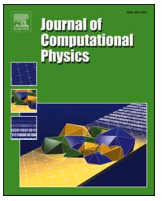
Journal of Computational Physics 542 (2025) 114373

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp



A novel unsupervised PINN framework with dynamically self-adaptive strategy for solid mechanics

Feiyang Wang [a], Wuzhou Zhai [b,*], Shuai Zhao [c], Jianhong Man [d]

[a] Department of Civil Engineering, College of Environmental Science and Engineering, Donghua University, No.2999 North Renmin Road, Shanghai 201620, China
[b] National Buried Infrastructure Facility (NBIF), School of Engineering, University of Birmingham, Edgbaston, Birmingham, UK
[c] Department of Civil and Environmental Engineering, The Hong Kong Polytechnic University, Hong Kong 999077, China
[d] School of Resources and Safety Engineering. University of Science and Technology Beijing, Beijing 100083, China

ARTICLE INFO

Keywords:
Physics-informed neural network
Neural network architecture
Augmented Lagrangian algorithm
Self-adaptive weight updating strategy
Solid mechanics
Information entropy

ABSTRACT

Unbalance in multi-task training has always posed significant challenges in deep learning, particularly for Physics-Informed Neural Networks (PINN). A novel unsupervised PINN framework configured with two neural network architectures has been developed for solid mechanics problems with pure boundary data. This framework features an innovatively formulated loss function, where loss weights are dynamically updated through a self-adaptive strategy using either the gradient normalization algorithm or the augmented Lagrangian algorithm, effectively tackling training imbalances across different types of boundary data. The PINN model consistently approximates solutions for solid mechanics problems with improved convergence and accuracy, as validated through a uniaxial tensile test case. The novel framework is robust and adaptable to two neural network architectures with different numbers of layers and neurons. About 30000 training epochs can reduce the prediction error of deformation to below 10^{-6}, meeting the requirements of computational solid mechanics. An information entropy of 1.3 bits, calculated from 500 well-trained PINN models, indicates that the novel unsupervised PINN framework exhibits low uncertainty. The findings uncover the so-called "squared rule" for selecting a suitable threshold in the convergence criterion. This study successfully addresses the critical scientific problem inherent in multitask learning, positioning PINN as a potentially universal and user-friendly method, offering an alternative for numerical computations.

1. Introduction

Machine learning, exemplified by Physics-informed neural networks (PINN), is powerful at capturing complex physical behavior, such as sharp transitions in Burgers' equation, while conventional numerical methods require substantial time and effort for temporal and spatial discretization to accurately handle localized mutations [1]. To improve the efficiency and accuracy, preprocessing and Batch Normalization (BN) layers are utilized to scale the training data in many traditional supervised learning scenarios, such as image recognition, prediction and inference. However, since PINN involves differential equations, standardizing the dataset may adversely affect the accuracy of derivatives. Therefore, it is not feasible to employ standardization techniques, which leads to low computational efficiency and accuracy of PINN. Consequently, constrained by computational efficiency and accuracy, PINNs have not yet been

* Corresponding author.
E-mail addresses: wangfy@dhu.edu.cn (F. Wang), w.zhai.1@bham.ac.uk (W. Zhai).

https://doi.org/10.1016/j.jcp.2025.114373
Received 9 September 2024; Received in revised form 7 July 2025; Accepted 11 September 2025
Available online 12 September 2025

extensively applied to solving classical linear and nonlinear partial differential equations (PDEs). Hence, there is an immediate necessity to construct a well-designed PINN framework, with the aim of improving its efficient and accurate, while broadening its applicability to complex physical problems.

Purely data-driven machine learning approaches are often physically inconsistent, owing to issues such as extrapolation or observational biases [2]. Therefore, prior physical information is often embedded into data-driven machine learning approaches. Karniadakis et al. (2021) classified the physical problems with available data into three possible categories, namely weak physics with big data, some physics with some data, and strong physics with small data. Correspondingly, machine learning methods can be summarized into three categories, as shown in Fig. 1. For category I, weak physics with big data, data-driven approaches are very powerful in discovering physical laws [3–5], making predictions and classifications [6], etc. For category II, a partial amount of observational data is available, while the physics is partially known, for example a given Navier-Stokes flow, but the boundary and geometry are not clear. Raissi et al. (2020) developed a hidden flow mechanics framework free from the geometry and boundary to learn the velocity and pressure fields with the observational concentration [7]. For the third category known as physics-informed deep learning, the governing physical law or underlying PDE is well known, along with small data stemming from the Neumann, Dirichlet or Robin boundary. The physics-informed deep learning has been extended to solid mechanics [8–10], fluid mechanics [11], thermodynamics [12,13] and other fields. As an illustrative example in thermodynamics [14,15], a thermodynamics-informed neural network (TINN) has been developed, achieving nearly a 20-fold acceleration in phase equilibrium calculations.

The physics-informed neural network (PINN) provides an alternative approach to solving various partial differential equations (PDEs), or other inverse problems for which traditional numerical methods are powerless. However, so far, PINN has not yet been able to outperform traditional numerical methods in terms of computational efficiency and accuracy when solving the classical linear and nonlinear partial differential equations [16]. In the early research, limited by the computing power, artificial neural networks are applied to solve PDE with a trial solution consisting of two parts [17]. The first part is fixed with the initial and boundary conditions and contains no adjustable parameters, and the second part is constructed so as not to affect the initial and boundary conditions. The loss function of PINN consists of PDEs, initial conditions (ICs), and boundary conditions (BCs). Soft constraints are usually adopted, but there is no guarantee that the approximate solution of PINN satisfies the ICs and BCs [18]. Hard-constraint frameworks have been developed to enforce the output of PINN to satisfy the ICs and BCs [19,20]. However, it is not easy to construct the hard-constraint framework for most applications. Liu et al. (2023) proposed a unified hard-constraint framework by the PDEs reformulated with extra fields [21]. Two PINN frameworks, X-TFC [22] and Deep-TFC [23], leverage the theory of functional connections [24] to analytically satisfy initial and boundary conditions, thereby eliminating the need to impose them as soft penalties in the loss function. In addition, the neural network architecture has been modified to improve the performance of PINN. A neural network architecture that treats the primary variables and the corresponding spatial gradients as outputs of separate neural networks works well in solid mechanics [25]. On this basis, Rezaei et al. (2022) developed a mixed-formulation PINN for solid mechanics, where the loss function only requires first-order derivatives [9].

However, PINNs applied to solid mechanics inherently function as multi-task neural networks, and maintaining balanced training across multiple physics-informed loss terms remains a significant challenge [26]. To address this challenge, many existing studies adopt loss weights to regulate the learning rate of individual task loss [27–32]. A more adaptive alternative is the Gradient Normalization (GradNorm) algorithm, which dynamically adjusts loss weights based on the initial loss magnitudes to balance the learning process for both regression and classification tasks [30]. However, this strategy depends heavily on the initial loss values, which are difficult, if not infeasible, to reliably estimate in complex solid mechanics problems due to their sensitivity to network initialization. To overcome this limitation, the GradNorm algorithm and the augmented Lagrangian method are modified and integrated with several neural network architectures. This self-adaptive weighting strategy, combined with the novel network designs, enables our unsupervised PINN framework to achieve faster convergence, higher accuracy, and improved robustness compared to conventional approaches.

Starting from this point, a novel PINN framework is developed for solid mechanics, specially aimed at enabling unsupervised learning. The remainder of this paper is organized as follows. Section 2 presents the novel unsupervised learning framework for PINN, including the neural network architecture, a novel loss function formulation, and the self-adaptive weight updating strategy, in detail.

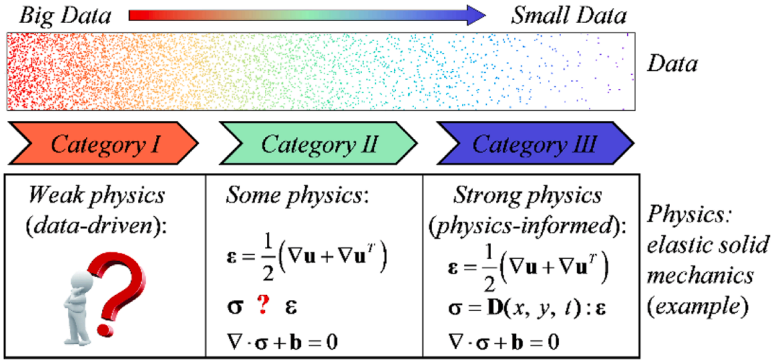


Fig. 1. Machine learning from data-driven to physics-informed.

In section 3, the developed PINN is applied to a solid mechanics problem, and validated by comparing it with the analytical solution. In section 4, the model's performance is thoroughly evaluated, including the sensitivity analysis of model parameters, assessment of model uncertainty and convergence criterion.

2. A novel unsupervised learning framework for PINN

2.1. Neural network architectures

A fully-connected neural network $\mathcal{N}^L(\mathbf{x})$ is used to map inputs $\mathbf{x}$ to outputs $\mathbf{u}$ with n_l neurons in the l-layers ($l = 1, 2, \ldots, L$). The transferring information between the layers l-1 and l is governed by $\mathcal{N}^l(\mathbf{x}) = \sigma(\mathbf{W}^l\mathcal{N}^{l-1}(\mathbf{x}) + \mathbf{b}^l)$, where σ is a nonlinear activation function, and $\mathbf{W}^l$ and $\mathbf{b}^l$ are the weight and bias of layer l. A hyperbolic tangent, tanh, is used as the activation function in the following work. With the above components in hand, the fully-connected neural network (FNN) for non-supervised PINN with multi-task is defined as

$$\text{input layer}: \mathcal{N}^0(\mathbf{x}) = \mathbf{x}, \left(\mathbf{x} \in \mathbf{R}^{\dim(\mathbf{x})}\right) \tag{1}$$

$$\text{hidden layer}: \mathcal{N}^l(\mathbf{x}) = \tanh\left(\mathbf{W}^l\mathcal{N}^{l-1}(\mathbf{x}) + \mathbf{b}^l\right), \ (l = 2, \ldots, L-1 \text{ or } l = 1, 2, \ldots, L-2) \tag{2}$$

$$\text{shared layer}: \mathcal{N}^l(\mathbf{x}) = \tanh\left(\mathbf{W}^l\mathcal{N}^{l-1}(\mathbf{x}) + \mathbf{b}^l\right), (l = 1 \text{ or } L-1) \tag{3}$$

$$\text{output layer}: \mathcal{N}^L(\mathbf{x}) = \left(\mathbf{W}^L\mathcal{N}^{L-1}(\mathbf{x}) + \mathbf{b}^L\right) \tag{4}$$

The neural network architectures for PINN are constructed using fully-connected neural networks (FNNs), with the coordinates $\mathbf{x}(x,$

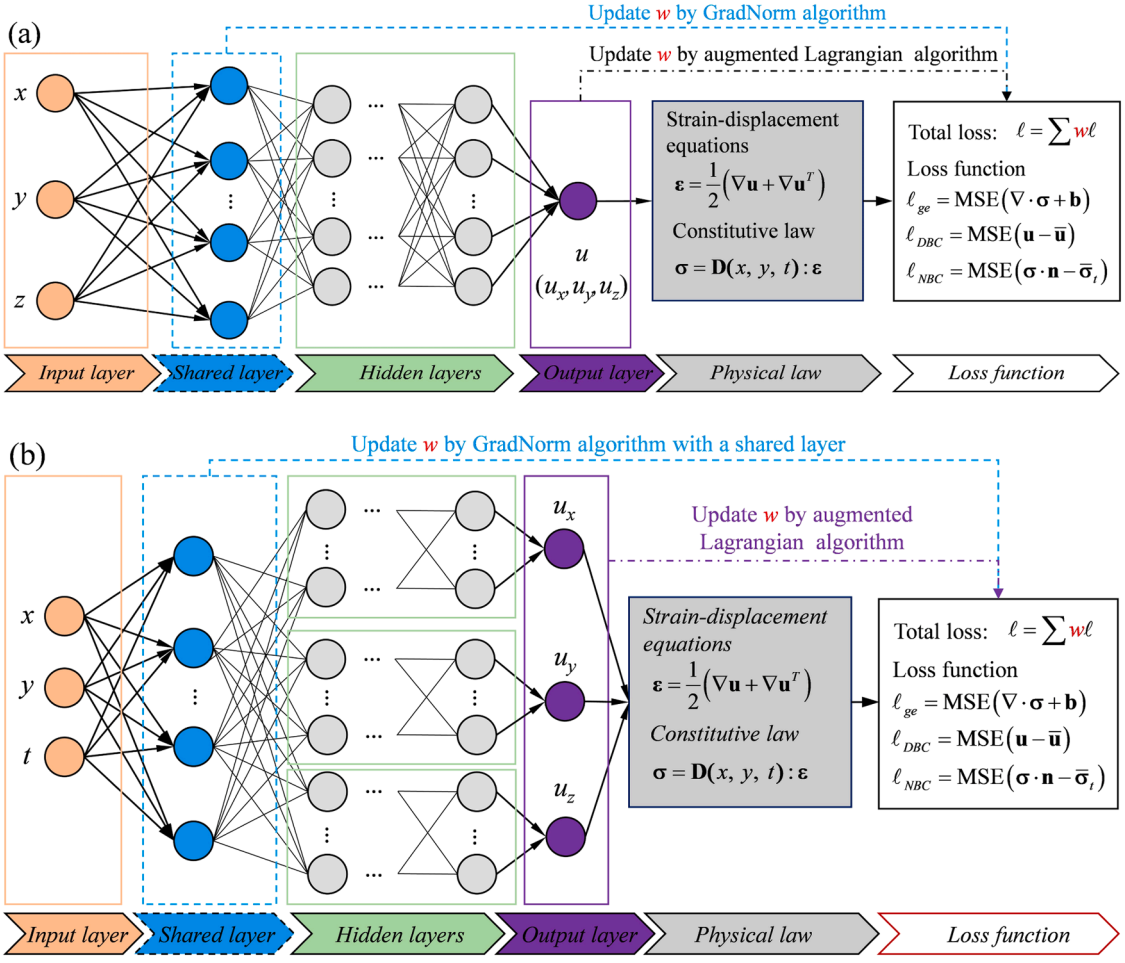


Fig. 2. Unsupervised PINN architectures for solid mechanics, (a) single neural network, (b) parallel neural network.

y, z) as the input and the primary unknowns, specially displacement $\mathbf{u}(u_x, u_y, u_z)$, as the output. In the PINN framework, the neural network is trained to approximate solutions to solid mechanics problems. As shown in Fig. 2, two distinct architectures are designed for the 2D and 3D solid mechanics, namely the single neural network and the parallel neural network. In the former configuration, a single neural network is responsible for evaluating all the primary unknowns. In contrast, in the parallel neural network architecture, each individual neural network is independently trained to approximate a single unknown. The loss function in the following section incorporates dynamically self-adaptive weights, which can be updated through two strategies applicable to both configurations. One self-adaptive weight updating strategy utilizes the Gradient normalization algorithm, relying on the gradients of the shared layer's weights, while the other is implemented using the augmented Lagrangian algorithm.

2.2. A novel formulation of loss function

In PINN, the loss function is intentionally crafted to enforce the fulfillment of the governing equations, boundary conditions, and initial conditions of the physical problem being addressed. During training, automatic differentiation via backpropagation is employed to compute the loss function, thus ensuring the robustness and generalizability of the PINN model in capturing the underlying physics. Liu et al. (2023) introduced a new form of the loss function, Eq. (5), by integrating the least squares method with the modified weighted residual method [33]. The new form of the loss function is written as follows.

$$\ell = \chi_g \int_{\Omega} R_g^2 \mathrm{d}\Omega + \chi_b \int_{\Gamma_b} R_b^2 \mathrm{d}\Gamma \tag{5}$$

where R_g and R_b are the residuals of the governing equation and the boundary condition (Dirichlet, Neumann, or mixed boundary conditions), χ_g and χ_b are the two scaling factors that enforce the residuals to have the same units and order of magnitude.

Although non-dimensionalization has mitigated substantial differences in the magnitude of each task's loss, the convergence issue still exists owing to training imbalance across different tasks. To address this challenge, the dynamically self-adaptive weights are introduced to maintain a balanced training process. An innovative formulation for the loss function is designed to incorporate these dynamically self-adaptive weights, allowing them to be automatically updated within the unsupervised PINN framework through either the Gradient normalization algorithm or the augmented Lagrangian algorithm.

$$\ell = \lambda_{\mathrm{g}}\chi_{\mathrm{g}} \int_{\Omega} R_{\mathrm{g}}^2 \mathrm{d}\Omega + \lambda_{\mathrm{n}}\chi_{\mathrm{n}} \int_{\Gamma_{\mathrm{n}}} R_{\mathrm{n}}^2 \mathrm{d}\Gamma + \lambda_{\mathrm{d}}\chi_{\mathrm{d}} \int_{\Gamma_{\mathrm{d}}} R_{\mathrm{d}}^2 \mathrm{d}\Gamma \tag{6}$$

where λ_{g}, λ_{n} and λ_{d} are the dynamically self-adaptive weights updated by the gradient normalization (GradNorm) algorithm or the augmented Lagrangian algorithm, χ_{g}, χ_{n} and χ_{d} are the corresponding dimensionless weights, and R_{n} and R_{d} are the residuals on the Neumann and Dirichlet boundary surfaces Γ_{n} and Γ_{d}.

2.3. Self-adaptive weight updating strategy

In multi-task PINNs, the losses primarily stem from the physical equations, initial and boundary condition data, as well as observational data. The training imbalance in PINNs arises due to the differing scales, units and convergence rates of these various loss terms associated with the governing physical equations, boundary conditions, and observational data. Conventionally, each component of the loss function is assigned an equal weight. However, since each task contributes differently to the total loss, tasks with larger initial loss magnitudes or faster gradients tend to dominate the training process, resulting in under-training of other tasks. This imbalance leads to inefficient optimization and compromises the overall convergence and accuracy of the PINN model. Consequently, it is crucial to implement an adaptive weight updating strategy that reflects the relative contribution of each task [28]. This strategy highlights the need for unsupervised learning within PINN to dynamically adjust the weight of individual tasks. Techniques such as the gradient normalization (GradNorm) algorithm (Chen et al., 2018 [30]) and the augmented Lagrangian algorithm (Lu et al., 2021 [34]) are powerful tools for dynamically updating the weights of tasks during PINN training, effectively tackling the issue of unbalanced loss gradients across multiple tasks.

2.3.1. Gradient normalization algorithm

The Gradient normalization (GradNorm) algorithm demonstrates robust performance by leveraging weights from a specific hidden layer, typically either the last or the first hidden layer, as illustrated in Fig. 2. This approach enables the algorithm to dynamically calculate the self-adaptive weights based on the gradients from the selected hidden layer, which can be formulated as an optimization problem.

$$\text{minimize} : \ell_{\text{grad}}(\omega) = \sum\nolimits_i |G_{Wi}(t) - \overline{G}_W(t)[r_i(t)]^{\alpha}|_1 \tag{7}$$

$$\text{subject to} : \sum_i \lambda_i = K, \lambda_i > 0 \tag{8}$$

where $G_{Wi}(t) = \| \nabla_W \lambda_i(t) L_i(t) \|_2$, $\overline{G}_W(t) = E(G_{Wi}(t))$, W is the weight of a shared layer in PINN, $L_i(t)$ refers to the loss of task i at training step t, $\widetilde{L}_i(t) = L_i(t)/L_i(0)$, $r_i(t) = \widetilde{L}_i(t)/E[\widetilde{L}_i(t)]$, α is the hyper-parameter, and K is the task number.

For solid mechanics, it is evident that the initial loss $L_i(0)$ is highly sensitive to the initialization of the PINN. However, with a well-configured neural network, it is reasonable to anticipate that the PINN will effectively approximate the partial differential equations (PDE). Consequently, the loss observed after training with a certain number of epochs appears to gradually diminish depending on the initialization of the PINN. A dynamic approach has been designed to ensure stable initial losses throughout the training process. This approach incorporates dynamically self-adaptive weights, which are periodically re-initialized at regular intervals, typically every n epochs. The pseudocode for unsupervised multi-task PINN implemented with the modified GradNorm algorithm is detailed in Table 1.

2.3.2. Augmented Lagrangian algorithm

In the PINN framework, the solid mechanics problem can be conceptualized as an optimization problem that aims to minimize a lost function subject to specific constraints.

$$\text{minimize} : g(\boldsymbol{\theta}) \tag{9}$$

$$\text{subject to} : b_i(\boldsymbol{\theta}) = 0 \quad (i = 1, 2, ..., n) \tag{10}$$

where $\boldsymbol{\theta}$**(W, b)** are the neural network's weights.

The augmented Lagrangian function for the solid mechanics problem is constructed by incorporating the governing physical equations, as well as arbitrary boundary and initial conditions. The optimization problem is thus represented by

$$L(\boldsymbol{\theta}, \lambda, c) = \lambda_g g(\boldsymbol{\theta}) + \sum_i^m \lambda_i b_i(\boldsymbol{\theta}) + \sum_i^m \frac{c}{2} \| b_i(\boldsymbol{\theta}) \|_2^2 \tag{11}$$

where $b_i(\boldsymbol{\theta})$ are the boundary constraints and initialization constraints, λ_i are the dynamically self-adaptive weights or associated Lagrange multipliers (with $\lambda_g = 1$ for the physical equation), and c is the regularization parameter or penalty parameter that helps enforce the boundary constraints.

With the augmented Lagrangian function established, the problem is transformed into an unconstrained optimization problem. During the (k+1)-th iteration, the neural network's parameters $\boldsymbol{\theta}$ can be updated using the following formulas.

$$\boldsymbol{\theta}^{k+1} = \arg\min_{\boldsymbol{\theta}} L(\boldsymbol{\theta}, \lambda^k, c^k) \tag{12}$$

For $\boldsymbol{\theta}^{k+1}$ to be the optimal solution, the gradient of the augmented Lagrangian function at $\boldsymbol{\theta}^{k+1}$ must be equal to zero.

$$\nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta}^{k+1}, \lambda^k, c^k) = 0 \tag{13}$$

The Lagrange multipliers are then updated by

$$\lambda_i{}^{k+1} = \lambda_i{}^k + c^k b_i(\boldsymbol{\theta}^{k+1}) \tag{14}$$

During each epoch, the neural network's parameters $\boldsymbol{\theta}$ and Lagrange multipliers are updated until the training satisfies the convergence criteria. The pseudocode for unsupervised multi-task PINN implemented with the augmented Lagrangian algorithm is

Table 1
Unsupervised PINN framework implemented with the modified GradNorm algorithm.

Algorithm 2	Modified GradNorm algorithm for unsupervised multi-task PINN
Preparation	Input: training data $\{(x, y)\}_N$
	Output: $\{(u, \varepsilon, \sigma)\}_N$, (displacement, strain and stress)
	Initialize PINN network for multi-task with a shared layer
	Pick a hyper-parameter $\alpha > 0$
Training	for *epoch* ← 1 to *N* do (*N*: epoch number)
	if *epoch* ← 1 or *epoch* % *n* ← 0 then
	shuffle data
	end
	compute task loss $l_i(t)$ $\forall$ *i*
	if *epoch* ← 1 or *epoch* % *n* ← 0 then
	initialize task loss weight $\lambda_i(t)$ $\forall$ *i*
	optimize task loss weight $\lambda_i(t)$ $\forall$ *i*
	end
	compute total loss $l = \Sigma_i \lambda_i(t) l_i(t)$
	compute $G_{Wi}(t)$ and $r_i(t)$ $\forall$ *i* (*W*, weight of shared layer)
	compute $\mathscr{L}_{\text{grad}} = \sum_i \lvert G_{Wi}(t) - \overline{G}_W(t)[r_i(t)]^\alpha \rvert_1$
	compute loss weight gradient $\nabla_\lambda L_{\text{grad}}$
	optimize task loss weight $\lambda_i(t+1)$ $\forall$ *i*
	if convergence criteria are satisfied
	stop the iteration
	end
	end

detailed in Table 2.

2.3.3. Application in neural network architectures

Both the modified GradNorm algorithm and the augmented Lagrangian algorithm can be seamlessly integrated within the single and parallel neural network architectures. During training, after the weights of the PINN are updated in each epoch, the dynamically self-adaptive weights are adjusted by the modified GradNorm algorithm based on the gradients of the shared layer's weights. Conversely, when using the augmented Lagrangian algorithm, the dynamically self-adaptive weights, functioning as Lagrange multipliers, are updated accordingly. This novel unsupervised learning framework for PINN effectively addresses the issue of training imbalance, thereby enhancing both accuracy and efficiency.

The unsupervised PINN framework is particularly designed to tackle the issue of imbalanced training, resulting in a significant improvement in both the convergence and accuracy of PINN models. The framework is innovative in the following aspects: (a) the introduction of non-dimensional weights ensures the consistency in the dimensionality and magnitude of each task's loss; (b) the self-adaptive weight updating strategy can automatically adjust the weights in real-time based on the contribution of each task during the training; (c) the designed neural network architectures is robustly adaptable to various weight updating algorithms.

3. Training and validation

The fundamentals of elastic solid mechanics are comprehensively reviewed, and the training data are prepared using the mechanical knowledge encompassing material properties, governing equations, as well as initial and boundary conditions. The novel loss function and training strategy for the PINN model are introduced in detail. The advantages of the innovative unsupervised learning framework are validated through a uniaxial tensile test case.

3.1. Overview of elastic solid mechanics

In elastic solid mechanics, there are three fundamental formulations that govern the behavior of solid mechanics. These formulations are the kinematic relation (displacement-strain relationship, Eq. (15)), constitutive relation (Hooke's law, Eq. (16)) and equilibrium partial differential equation (PDE, Eq. (17)).

$$\boldsymbol{\varepsilon} = \frac{1}{2}\left(\nabla \mathbf{u} + \nabla \mathbf{u}^T\right) \tag{15}$$

$$\boldsymbol{\sigma} = \mathbf{D} : \boldsymbol{\varepsilon} \tag{16}$$

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = 0 \tag{17}$$

where $\mathbf{u}$ is the displacement, $\boldsymbol{\sigma}$ and $\boldsymbol{\varepsilon}$ are the stress and strain, $\mathbf{D}$ is the stiffness, and $\mathbf{b}$ is the body force per unit volume.

Boundary conditions are the main components of training data for PINN when solving physical problems in elastic solid mechanics. There are three common types of boundary conditions, Dirichlet boundary condition, Neumann boundary condition and mixed boundary condition. Taking the uniaxial tensile plate depicted in Fig. 3 as an example, the Dirichlet, Neumann, and mixed boundary conditions are specified as follows.

Table 2
Unsupervised PINN framework implemented with the augmented Lagrangian algorithm.

Algorithm 2	Augmented Lagrangian algorithm for unsupervised multi-task PINN
Preparation	Input: training data $\{(x, y)\}_N$
	Output: $\{(u, \varepsilon, \sigma)\}_N$, (displacement, strain and stress)
	Initialize PINN model for multi-task with a shared layer
	Initialize Lagrange multipliers λ and penalty parameter $c > 0$
Training	for *epoch* $\leftarrow$ 1 to N do (N: epoch number)
	if *epoch* $\leftarrow$ 1 or *epoch* % $n \leftarrow$ 0 then
	shuffle data
	end
	compute model output u
	compute the loss of the augmented Lagrangian function: $L(\boldsymbol{\theta}, \lambda, c) = \lambda_g g(\boldsymbol{\theta}) + \sum_i^m \lambda_i b_i(\boldsymbol{\theta}) + \sum_i^m \frac{c}{2} \| b_i(\boldsymbol{\theta}) \|_2^2$
	update neural network's parameters $\boldsymbol{\theta}$
	update the Lagrange multipliers $\lambda_i \; \forall \; i$ and penalty parameter c
	compute each task loss $l_i(t) \; \forall \; i$
	if convergence criteria are satisfied
	stop the iteration
	end
	end

$$\begin{cases} u_x(x=0,y)=0 \\ u_y(x,y=0)=0 \end{cases} \tag{18}$$

$$\begin{cases} \tau_{xy}(x=0,y)=\tau_{xy}(x=1,y)=0, \sigma_x(x=1,y)=0 \\ \tau_{xy}(x,y=0)=\tau_{xy}(x,y=1)=0, \sigma_y(x,y=1)=p \end{cases} \tag{19}$$

where (x, y) are the coordinates within the plate, u_x and u_y are the displacement components on the Dirichlet boundary, τ_{xy} and σ_x are the stress components on the Neumann boundary, and p is the applied load.

3.2. Details of the neural network architectures

The configuration of the novel unsupervised learning framework for PINN includes the neural network architectures, the number of layers and the number of neurons. Two neural network architectures, the single neural network and parallel neural network shown in Fig. 2, are trained to conduct thorough performance evaluation. Specifically, the number of hidden layers and the number of neurons in these two neural networks are set to different values. Table 3 provides several neural network architectures, including the number of hidden layers, the number of neurons within each hidden layer, and the total number of training parameters.

3.3. Data preparation

The training data for PINN in solid mechanics involves the knowledge of material properties, governing equations, initial and boundary conditions, as well as observational data. These components are employed to formulate a loss function, as depicted in Eq. (6), which serves as the guiding principle for training the neural network to approximate the solution of the underlying PDE. The material properties are crucial in the training of PINN because they define the way in which the material reacts to the external loads. Input material parameters, including Young's modulus, Poisson's ratio, and other parameters are incorporated in the loss function of PINN when tackling both forward and partial inverse problems in the field of solid mechanics. This study primarily concentrates on the forward problem in solid mechanics with the novel unsupervised PINN framework. For the training case represented in Fig. 3, the material parameters and load in the PINN model are specified as follows: Young's modulus $E = 20,000$ kPa, Poisson's ratio $\mu = 0.2$, and the applied load $p = 10$ kPa.

The training data for governing equations refers to the positional information or collocation points within the computational domain. Two distinct strategies are available for generating the coordinates of collocation points within the computational domain, uniform spacing sampling and random sampling. In the uniform spacing sampling strategy, collocation points are methodically placed at regular intervals across the computational domain, while in the random sampling strategy, collocation points are selected following a uniform distribution throughout the computational domain, as shown in Fig. 4 (a) and (b).

The training data for initial and boundary conditions includes the position information, and the known quantities such as displacement, stress or velocity in dynamic problems. In solid mechanics, the initial conditions are typically imposed on the computational domain at the initial time, while the boundary conditions are typically prescribed for the boundary surfaces. The training data for the initial and boundary conditions can be achieved through uniform spacing sampling or random sampling. Uniform spaced and uniform distributed data points are shown in Fig. 4 (c) and (d), respectively.

In some scenarios, data are collected or measured at specific points within the computational domain, either through numerical simulation or experiments. These data can serve as valuable resource for improving PINN model training by constraining the model's predictions in line with the observed behavior. In this study, finite element (FE) simulations are employed as a benchmark to evaluate the accuracy of the PINN model.

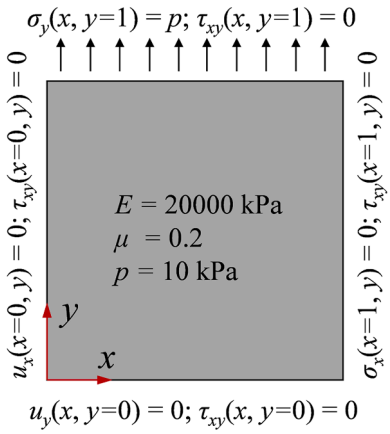


Fig. 3. Geometry and boundary conditions of a uniaxial tensile plate.

Table 3
Neural network architectures.

Architecture	Network number	Hidden layer number	Neurons	Training parameter number
Single neural network	S11	5	30	3872
	S12	5	50	10452
	S13	5	70	20232
	S14	5	90	33212
	S21	3	50	5352
	S22	5	50	10452
	S23	7	50	15552
	S24	9	50	20652
Parallel neural network	P11	5	30	7592
	P12	5	50	20652
	P13	5	70	40112
	P14	5	90	65972
	P21	3	50	10452
	P22	5	50	20652
	P23	7	50	30852
	P24	9	50	41052

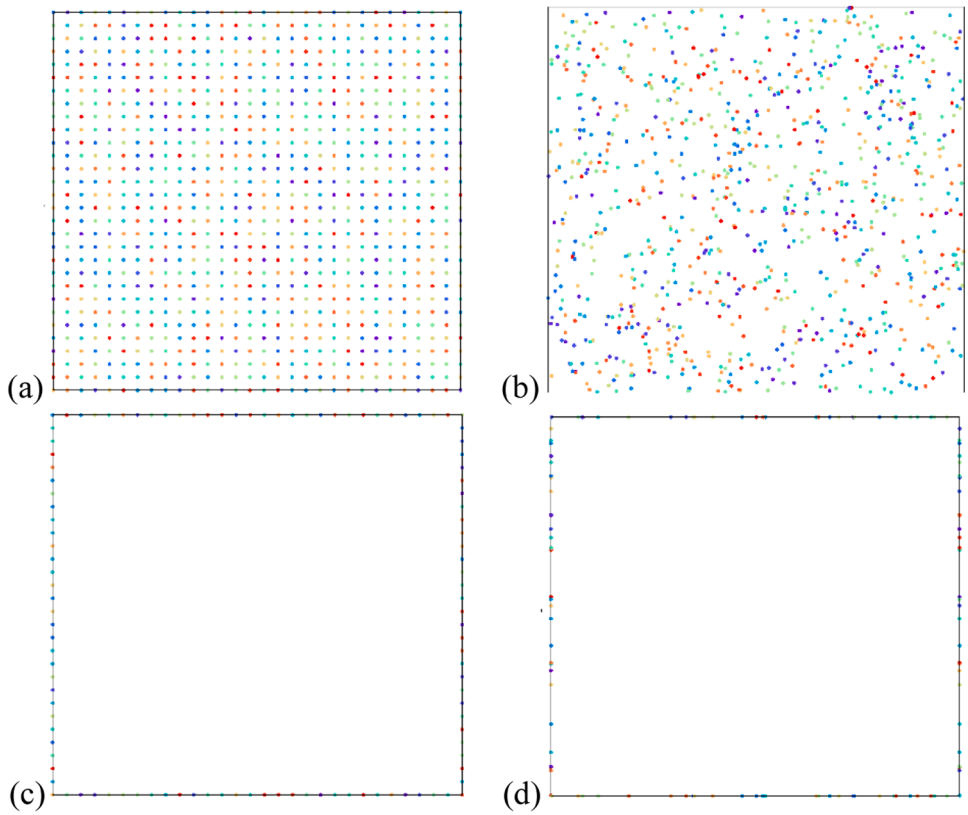


Fig. 4. Training data for PINN of a uniaxial tensile plate, (a) uniform collocation points, (b) random collocation points, (c) uniform data points on boundary surfaces, (d) random data points on boundary surfaces.

3.4. Model training

3.4.1. Loss function

The loss function is formulated to ensure that the PINN model adheres to the governing equations or PDEs, boundary conditions and initial conditions of the solid mechanics problems. In PINNs for solid mechanics, the Mean Squared Error (MSE) is often chosen to calculate the loss function [25,35]. With the available training data, the loss function in Eq. (6) can be rewritten in the following form.

$$\ell = \ell_{\mathrm{g}} + \ell_{\mathrm{n}} + \ell_{\mathrm{d}} = \lambda_{\mathrm{g}}\chi_{\mathrm{g}}^2 A_{\mathrm{g}} MSE_{\mathrm{g}} + \lambda_{\mathrm{n}}\chi_{\mathrm{n}}^2 L_{\mathrm{n}} MSE_{\mathrm{n}} + \lambda_{\mathrm{d}}\chi_{\mathrm{d}}^2 L_{\mathrm{d}} MSE_{\mathrm{d}} \tag{20}$$

where λ and χ represent the dynamically self-adaptive weight and dimensionless weight, as elaborated after Eq. (6), and A, L denote the area of the computational domain and the length of the boundary. The subscripts g, n and d correspond to quantities associated with the governing equation, Neumann boundary and Dirichlet boundary, respectively. For instance, MSE_g specifically denotes the mean squared error for the governing equation.

The mean square errors for the governing equation, Neumann boundary and Dirichlet boundary can be expressed as follows.

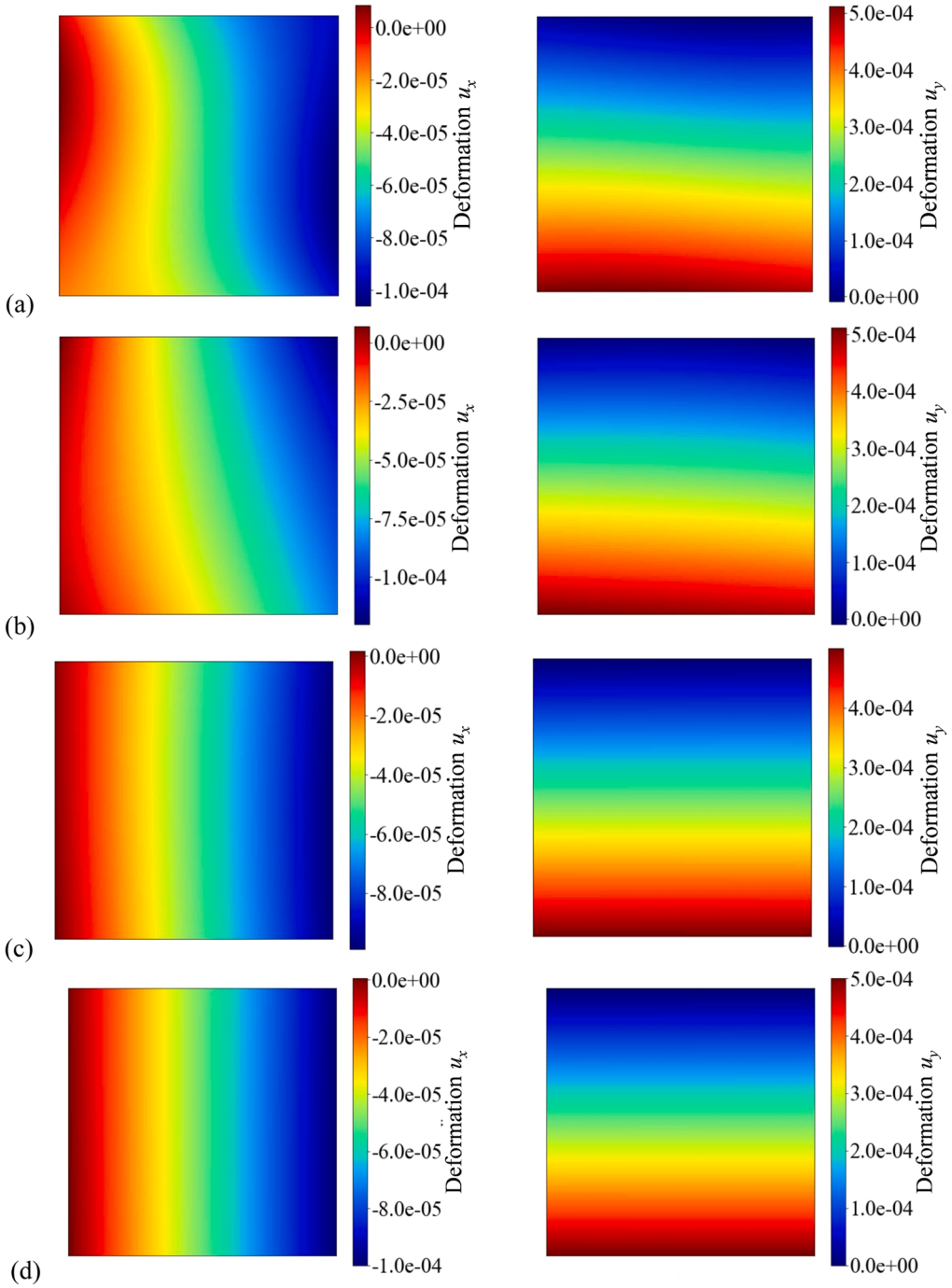


Fig. 5. Contours of deformation, u_x and u_y, predicted by the proposed PINN model for different predefined thresholds, (a) 1×10^{-9}, (b) $1 \times \times 10^{-10}$, (c) 1×10^{-11}, (d) 1×10^{-12}.

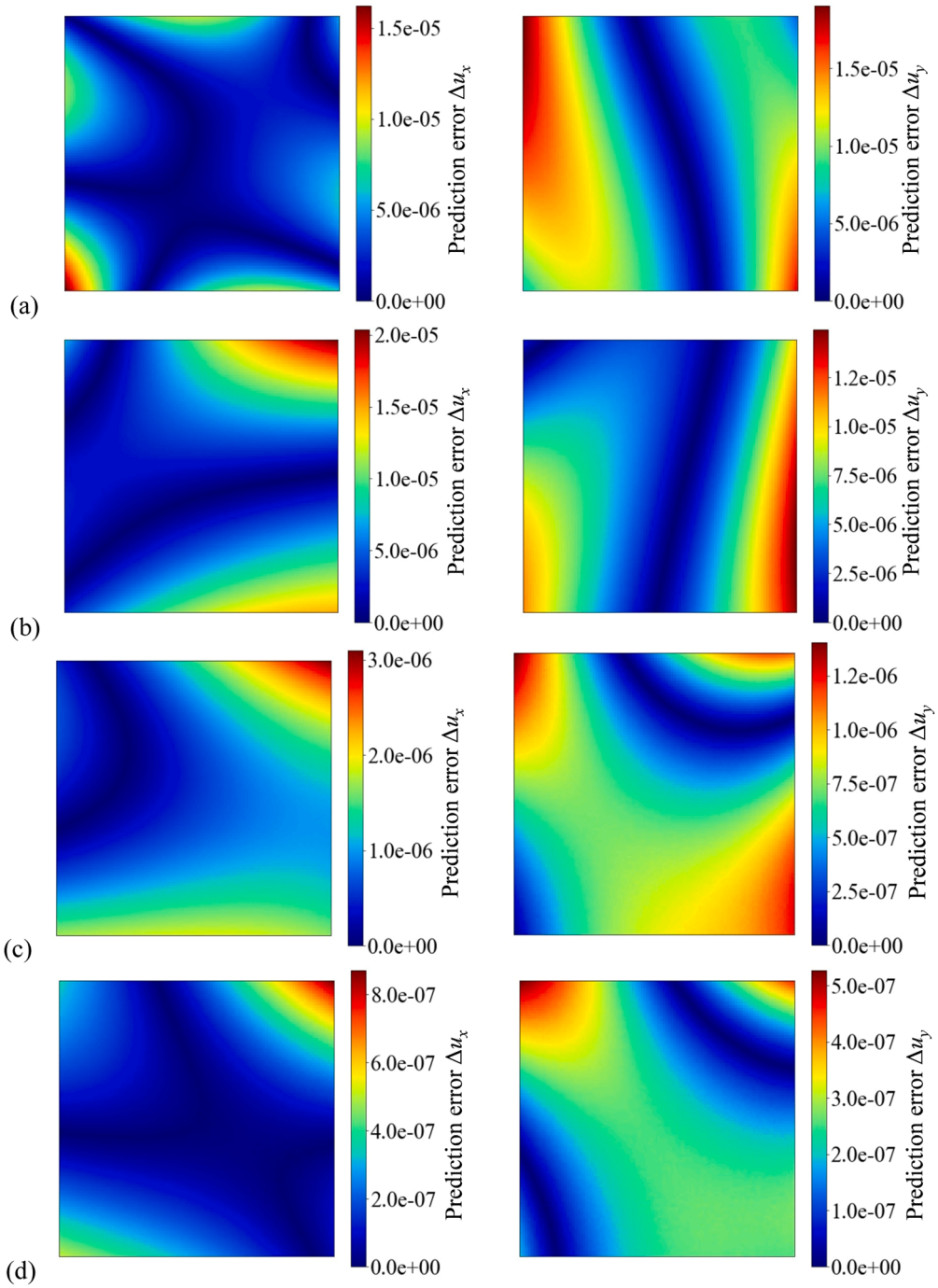


Fig. 6. Prediction error in deformation, Δu_x and Δu_y, of the proposed PINN model under different predefined thresholds, (a) 1×10^{-9}, (b) 1×10^{-10}, (c) 1×10^{-11}, (d) 1×10^{-12}.

$$\begin{cases} MSE_{\mathrm{g}} = \dfrac{1}{N_{\mathrm{g}}} \sum_{i=1}^{N_{\mathrm{g}}} (\nabla \cdot \boldsymbol{\sigma}_i + \mathbf{b}_i)^2 \\ MSE_{\mathrm{n}} = \dfrac{1}{N_{\mathrm{n}}} \sum_{j=1}^{N_{\mathrm{n}}} (\boldsymbol{\sigma}_j \cdot \mathbf{n}_j - \overline{\boldsymbol{\sigma}}_j)^2 \\ MSE_{\mathrm{d}} = \dfrac{1}{N_{\mathrm{d}}} \sum_{k=1}^{N_{\mathrm{d}}} (\mathbf{u}_k - \overline{\mathbf{u}}_k)^2 \end{cases} \tag{21}$$

where N_{g}, N_{n} and N_{d} are the number of collection points, the number of data points on Neumann boundary, and the number of data points on Dirichlet boundary, respectively.

There is often a notable difference between the losses associated with PDEs and boundary conditions due to dimensional discrepancies. To address this issue, dimensionless weights, χ_{g}, χ_{n} and χ_{d}, are employed to mitigate the differences in their orders of magnitude.

$$\chi_{\mathrm{g}} = \frac{l}{E} : \chi_{\mathrm{n}} = \frac{1}{E} : \chi_{\mathrm{d}} = \frac{1}{l_{\mathrm{d}}} \tag{22}$$

where l is the side length of the computational domain, E is the elastic modulus, l_{d} is the length of Dirichlet boundary.

3.4.2. Training strategy

Given the limited size of the training dataset for solid mechanics problems, a full-batch training strategy is employed for all neural networks. In this training strategy, the training data is fed into the neural network with the full batch size during every epoch. To mitigate overfitting to the data order, the training data is periodically reshuffled at a specified interval, such as a certain number of epochs. However, for the reproducibility of the well-trained PINN model, shuffling is disabled during training, and the torch.manual_seed() function is initialized at the start of the program to generate consistent random number sequences across different runs.

During the training process, selecting a suitable optimizer is essential to effectively adjust the model's parameters to minimize the defined loss function. Notably, gradient-based optimizers like optimizers Adam [36] and L-BFGS-B [37] are commonly employed for this purpose. L-BFGS-B has been successfully applied to various problems, including 1D, 2D, 3D stretching problem [8], as well as the Poisson and diffusion-advection systems [18]. Adam has proven to be a preferred choice for handling linear elasticity problems [25] as well as for simulating high-speed aerodynamic flows [38]. For small-scale training data, the L-BFGS-B optimizer is generally recommended, while the Adam optimizer is more suited for deep learning-based PINNs [39]. In the novel unsupervised PINN framework, the Adam optimizer is employed to optimize the model's parameters. The dynamically self-adaptive weights in Eq. (20) are determined through either the modified GradNorm algorithm or the augmented Lagrangian method, both of which have been introduced in detail in the previous section.

3.5. Model validation

The stopping criterion for the PINN model is configured using the early stopping technique, which aims to achieve a stable state where further training no longer yields significant performance gains. This technique is widely adopted to optimize training duration and prevent the risk of overfitting. Convergence is monitored by assessing changes in loss values over a specific number of epochs. The training is considered converged if each loss term in Eq. (20) falls below a predefined threshold for a specified number of consecutive epochs. To maintain the stability of the trained model, the number of consecutive epochs is fixed at 100. The predefined thresholds are set to 1.0×10^{-9}, 1.0×10^{-10}, 1.0×10^{-11}, and 1.0×10^{-12}, respectively.

The contours of deformation u_x and u_y, predicted by the well-trained PINN model configured with a parallel neural network

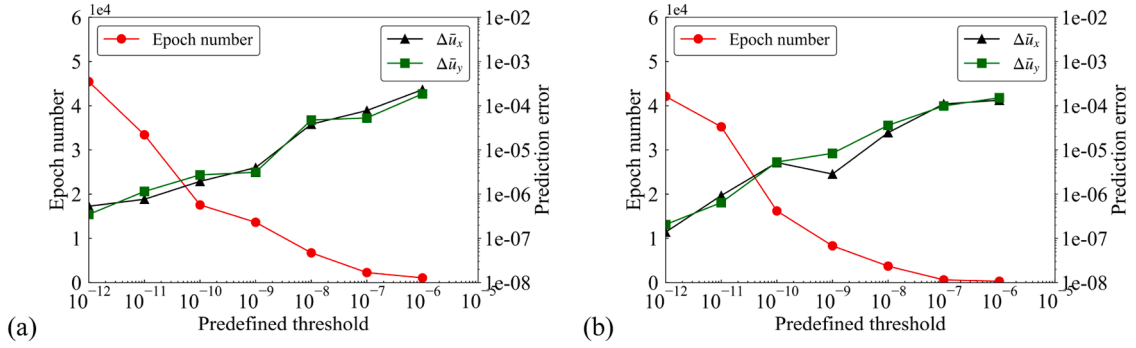


Fig. 7. Curves of training epoch number and prediction error varying with predefined threshold, (a) single neural network, (b) parallel neural network.

containing 5 hidden layers, each with 50 neurons per layer, are illustrated in Fig. 5. The hyperparameters, the weighting factor α in the GradNorm algorithm and the penalty coefficient c in the augmented Lagrangian method, are empirically selected to achieve stable training and fast convergence after several preliminary tests. Specifically, $\alpha = 0.2$ and $c = 0.5$ are consistently adopted across all cases. Fig. 6 shows the prediction error of deformation Δu_x and Δu_y, quantified as the absolute error between the PINN model and the analytical solution. The prediction error demonstrates a notable improvement in the accuracy of the model, as the predefined threshold decreases from 1.0×10^{-9} to 1.0×10^{-12}. After conducting a series of trails, it has been determined that setting the predefined threshold to 1.0×10^{-11}, the PINN model yields an average prediction error $\Delta\overline{u}_x$ of 9.3×10^{-7} m, far exceeding the engineering computational accuracy requirements.

The logarithm of the average prediction error tends to be approximately linear with respect to the logarithm of the predefined threshold, which can be observed in both the single neural network and the parallel neural network, as shown in Fig. 7. For the same predefined threshold, the single neural network requires slightly more training epochs, but only half the model parameters, which means less computational power. When the predefined threshold exceeds 1.0×10^{-11}, the number of training epochs increases dramatically, with a significant increase in computational power requirement and only a slight improvement in accuracy. Therefore, in the subsequent PINN model training, the predefined threshold is consistently set to 1.0×10^{-11} to optimize the accuracy and efficiency.

3.6. Advantages of the novel PINN framework

A parallel neural network structure is employed to conduct a comparative analysis between the proposed PINN framework and a baseline PINN framework without dynamically self-adaptive weights. The proposed PINN framework incorporates the augmented Lagrangian algorithm to dynamically adjust the contributions of different loss components. Variations in the order of magnitude of the elastic modulus E exacerbate the imbalance among the loss components corresponding to Dirichlet boundaries, Neumann boundaries and governing equations. To assess model accuracy and efficiency, training is independently conducted for elastic modulus $E = 200$ kPa, 2,000 kPa, and 20,000 kPa, while maintaining all other parameters consistent with the validation model. A predefined convergence threshold of 1.0×10^{-11} is enforced to ensure consistent accuracy across all training scenarios.

To eliminate hardware-dependent biases (e.g., GPU), computational efficiency is measured by the number of training epochs required to achieve the predefined accuracy. As illustrated in Fig. 8 (a), at E =2000 kPa, the proposed PINN model achieves the target accuracy using only 50% of the epochs required by the baseline model. Notably, at E =200 kPa, shown in Fig. 8 (b), the proposed PINN model converges within 6×10^5 epochs, while the baseline model fails to converge and exhibits oscillatory behavior even after 6×10^6 epochs. These results demonstrate that the proposed PINN framework, with its dynamically self-adaptive weighting strategy, significantly enhances training efficiency, improves prediction accuracy, and guarantees convergence stability, particularly under conditions of multi-task loss imbalance.

4. Model performance assessment

4.1. Sensitivity analysis of PINN model parameters

Several model parameters, encompassing learning rate, data number, layer number, neuron number, can influence both the training efficiency and prediction accuracy of the PINN model. To refine our selection of these parameters, the PINN models are configured with single neural network and parallel neural network architectures, and are separately trained using various values for each parameter. In the following model training, a set of baseline parameters is used, 5 hidden layers, 50 neurons per layer, a learning rate of 1×10^{-3}, and data point spacing of 0.01 m (equivalent to 100 data points in each direction). By systematically tuning these parameters, we aim to clarify their influence on the model's overall performance and provide guidelines for optimal parameter settings.

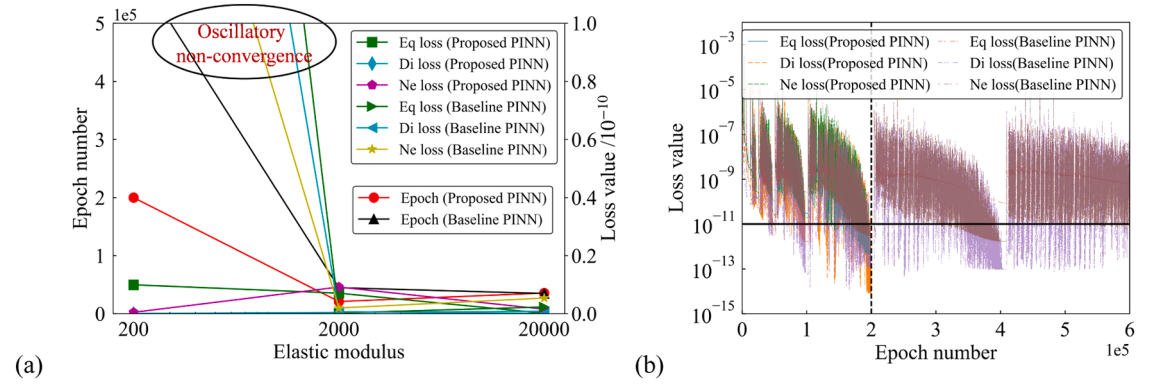


Fig. 8. Comparative analysis of training losses: proposed PINN model vs baseline PINN model, (a) training efficiency at the same accuracy level, (b) curves of loss value with epoch number.

For each model parameter, the training epoch number and prediction error ($\Delta\bar{u}_x$ and $\Delta\bar{u}_y$) serve as pivotal metrics to evaluate both the training efficiency and the prediction accuracy. Fig. 9 shows the curves of the epoch number and prediction error with the concerned parameter for the PINN model configured with a single neural network architecture. The training fails to converge when the learning rate exceeds 3.9×10^{-3}, whereas the epoch number sharply increases when the learning rate is below 1.0×10^{-4}. In terms of model efficiency, the influence of parameters from high to low is learning rate, layer number, and neuron number. The spacing of data points has a minimal effect on the number of epochs required for training. However, denser data points demand more computational power. With the same predefined threshold of 1.0×10^{-11}, well-trained PINN models achieve approximate prediction accuracy.

Fig. 10 shows the curves of the epoch number and prediction error with the concerned parameter of the PINN model configured with a parallel neural network architecture. The influence of model parameters on this architecture is similar to that on the single neural network architecture. Overall, the parallel architecture with suitable numbers of layers or neurons offers better computational efficiency and fewer fluctuations. The training fails to converge when the learning rate exceeds 3.0×10^{-3}, whereas the epoch number sharply increases when the learning rate is below 1.0×10^{-4}. Haghighat et al. (2021) observed that deeper neural network architectures yield less accurate weights, owing to redundancy in the layers or neurons. The developed neural network architectures exhibit robust adaptability to varying numbers of layers or neurons, as demonstrated in Figs. 9 and 10. However, it is crucial to note that an excessive number of layers or neurons can result in an unnecessary consumption of computational power. Therefore, considering both the efficiency and accuracy, it is recommended that the learning rate, data point spacing, layer number, and neuron number for both types of PINN models should be within the ranges of $1.0 \times 10^{-3} \sim 3.0 \times 10^{-3}$, $1.0 \times 10^{-2} \sim 4.0 \times 10^{-2}$ m, 4~10 layers, 40~100 neurons, respectively.

4.2. An information entropy-based uncertainty assessment

The observed fluctuations in the accuracy of the previously mentioned well-trained PINN models can be traced back to the inherent uncertainty in the training process. This uncertainty arises naturally from the incorporation of randomness in weight initialization, data shuffling, and other stochastic factors, leading to diverse outcomes with different levels of efficiency and accuracy. Without setting a fixed seed for the random number generator, these factors inherently introduce uncertainty into the PINN model, complicating the assessment of its performance. To comprehensively understand the model's performance, the concept of information entropy is adopted to quantify these uncertainties [40]. The parallel neural network architectures for the PINN model consisting of 5 layers, each containing 50 neurons, is trained at a learning rate of 1.0×10^{-3} with data point spaced at 1.0×10^{-2} m.

500 PINN models are individually trained for each predefined threshold, and the prediction errors are statistically analyzed to calculate the information entropy $H(\cdot)$ at position $\mathbf{x}$ using Eq. (23). The information entropy for each predefined threshold is computed using the corresponding 500 well-trained PINN models. The information entropy contours are then illustrated in Fig. 11, and the

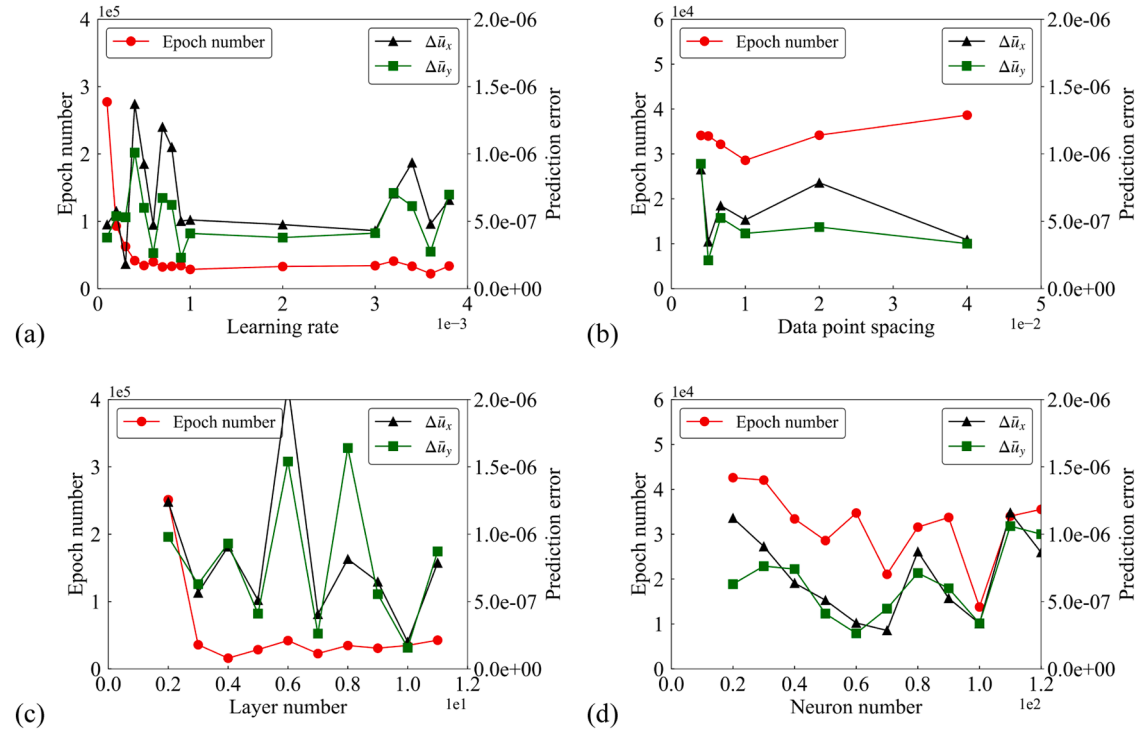


Fig. 9. Curves of the epoch number and prediction error with the concerned parameter for the PINN model configured with a single neural network architecture, (a) learning rate, (b) data number, (c) layer number, (d) neuron number.

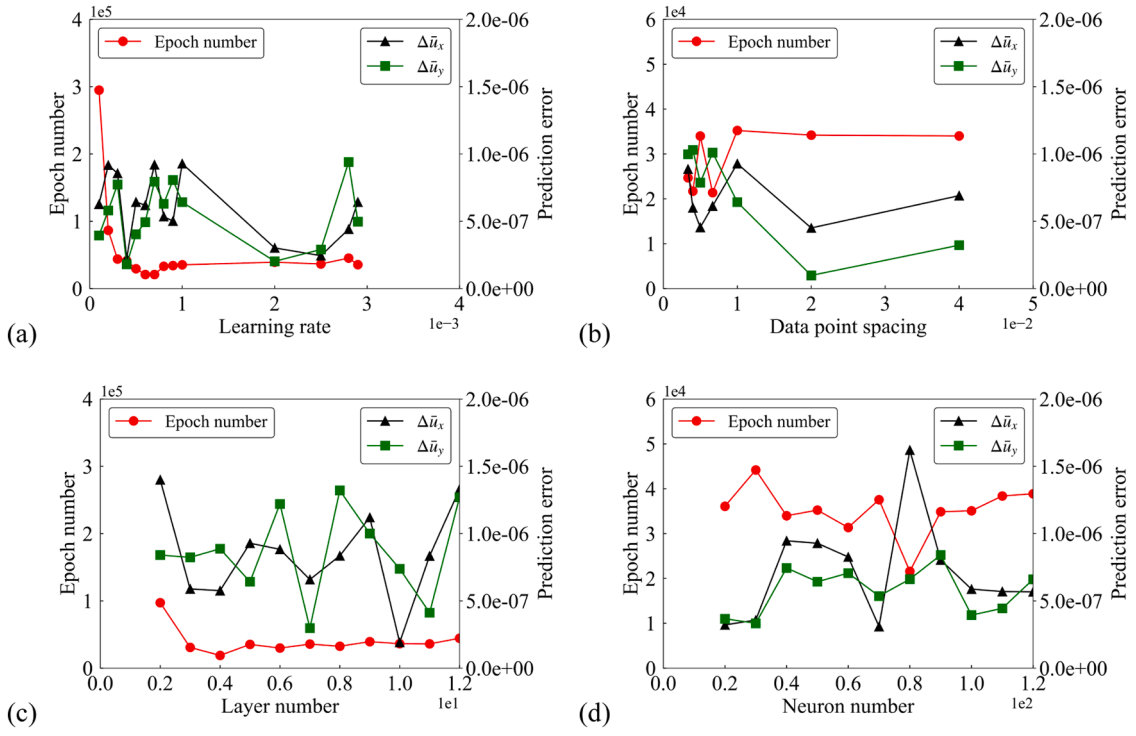


Fig. 10. Curves of the epoch number and prediction error with the concerned parameter for PINN model configured with a parallel neural network architecture, (a) learning rate, (b) data number, (c) layer number, (d) neuron number.

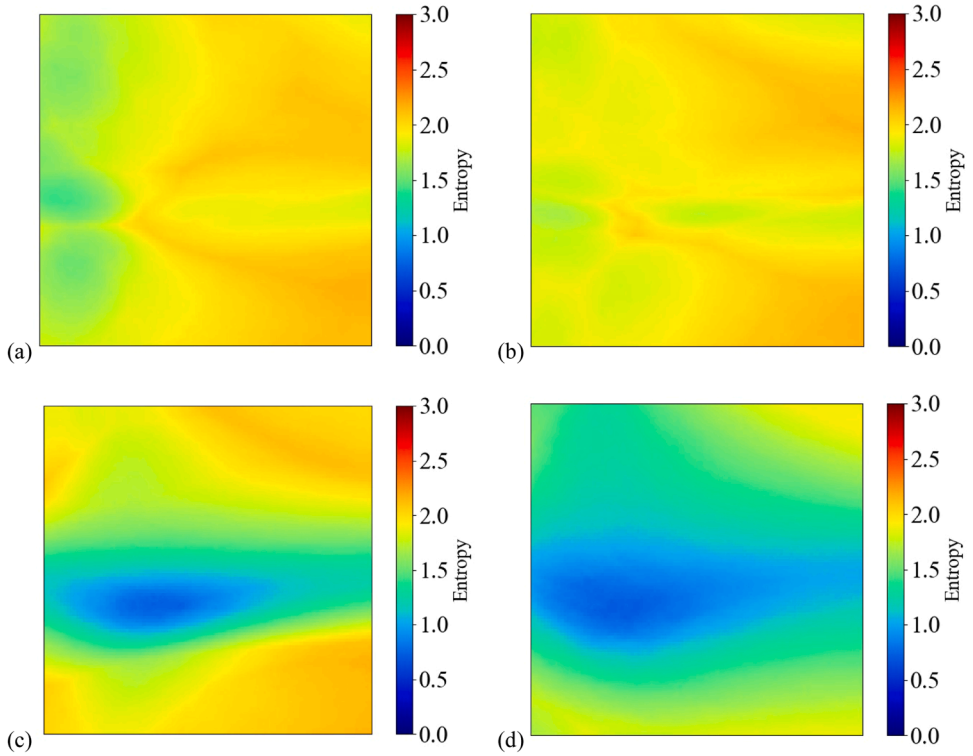


Fig. 11. Information entropy contours of well-trained PINN model under different predefined thresholds, (a) 1×10^{-8}, (b) 1×10^{-9}, (c) 1×10^{-10}, (d) 1×10^{-11}.

information entropy histogram in the computational domain is plotted in Fig. 12.

$$H(\Delta u^{\mathbf{x}}) = -\sum_{i=1}^{n} P(\Delta u_i^{\mathbf{x}}) \log P(\Delta u_i^{\mathbf{x}}) \tag{23}$$

where $\Delta u^{\mathbf{x}}$ is the prediction error at the position $\mathbf{x}$, $H(\cdot)$ represents the corresponding information entropy ($0 < H(\cdot) < \log_2 n$), n is the number of intervals computed by $n = 1 + \log_2 500 \approx 10$ in this benchmark, and $P(\cdot)$ denotes the corresponding probability [41]).

As the predefined threshold of the convergence criterion decreases gradually, the information entropy correspondingly shifts towards smaller values, indicating a reduction in the uncertainty of the trained PINN model. Specifically, when the predefined threshold reaches 1×10^{-11}, the average information entropy decreases to 1.31 bits, and the distribution of information entropy is approximately normal. By comprehensively considering the prediction error contours in Fig. 6 and the information entropy contours in Fig. 11, it is evident that the information entropy of 1.31 bits signifies high prediction accuracy and low uncertainty of the PINN model. Therefore, the low information entropy value demonstrates the consistency and stability of the proposed PINN framework under varying initializations and training conditions.

4.3. Convergence criterion

The developed PINN is applied to the solid mechanics problem illustrated in Fig. 3, considering various load conditions. To balance computational accuracy and efficiency, approximate convergence criteria should be selected for different load conditions. The training of the PINN model are conducted under different predefined thresholds. For load magnitudes p of 10^0 kPa, 10^1 kPa, 10^2 kPa and 10^3 kPa, the curves of the relative prediction error of vertical deformation and predefined threshold are shown in Fig. 13.

For loads p of 10^0 kPa, 10^1 kPa, 10^2 kPa and 10^3 kPa, when the relative prediction error is 1 %, the corresponding predefined thresholds should not exceed 10^{-13}, 10^{-11}, 10^{-9} and 10^{-7}, respectively. The contours of vertical deformation and its relative prediction error are illustrated in Figs. 14 and 15. It is evident that these predefined thresholds ensure accurate computational results consistent with physical laws. The results reveal a "squared rule", meaning that at a relative prediction error of 1 %, the reduction factor of the predefined threshold is approximately the square of the load increase factor. This rule lays a foundation for selecting a suitable threshold for the convergence criterion, facilitating the application and scalability of the developed PINN framework in practice.

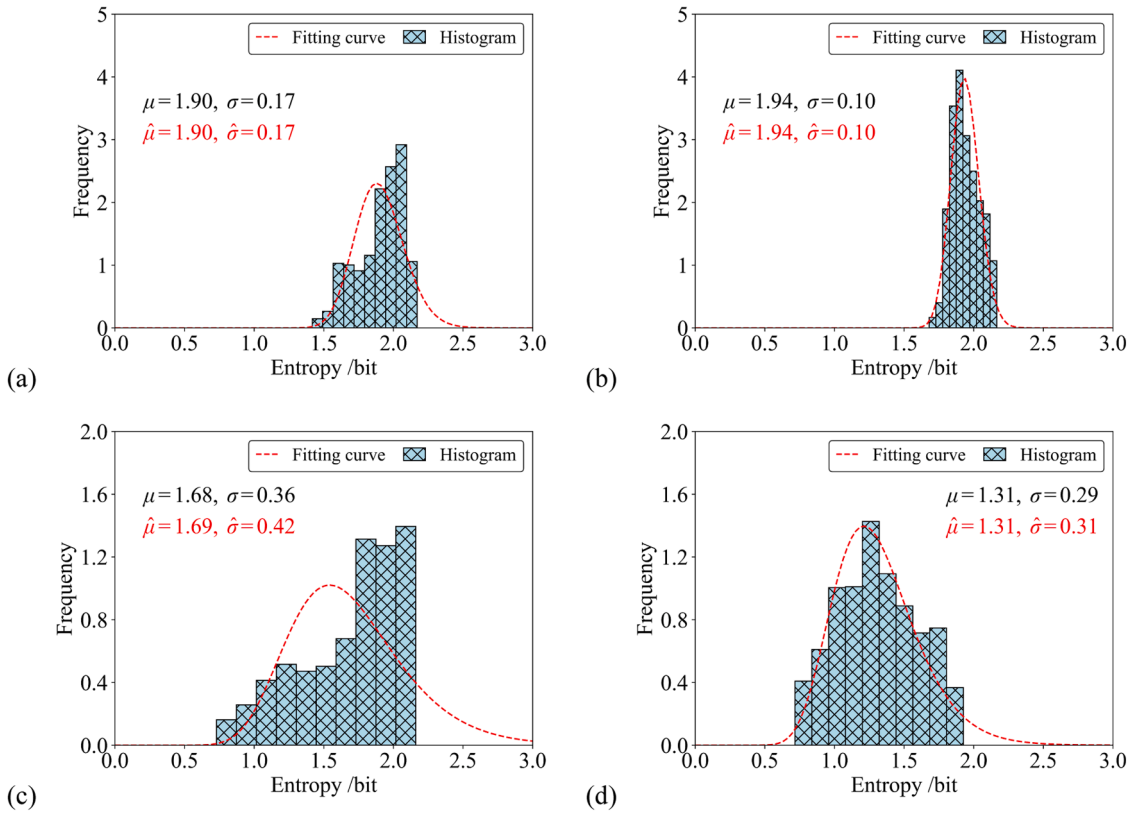


Fig. 12. Information entropy histogram of well-trained PINN model under different predefined thresholds, (a) 1×10^{-8}, (b) 1×10^{-9}, (c) 1×10^{-10}, (d) 1×10^{-11}.

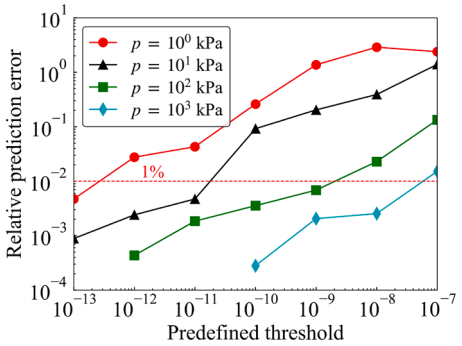


Fig. 13. Curves of the relative prediction error and predefined threshold under different loads.

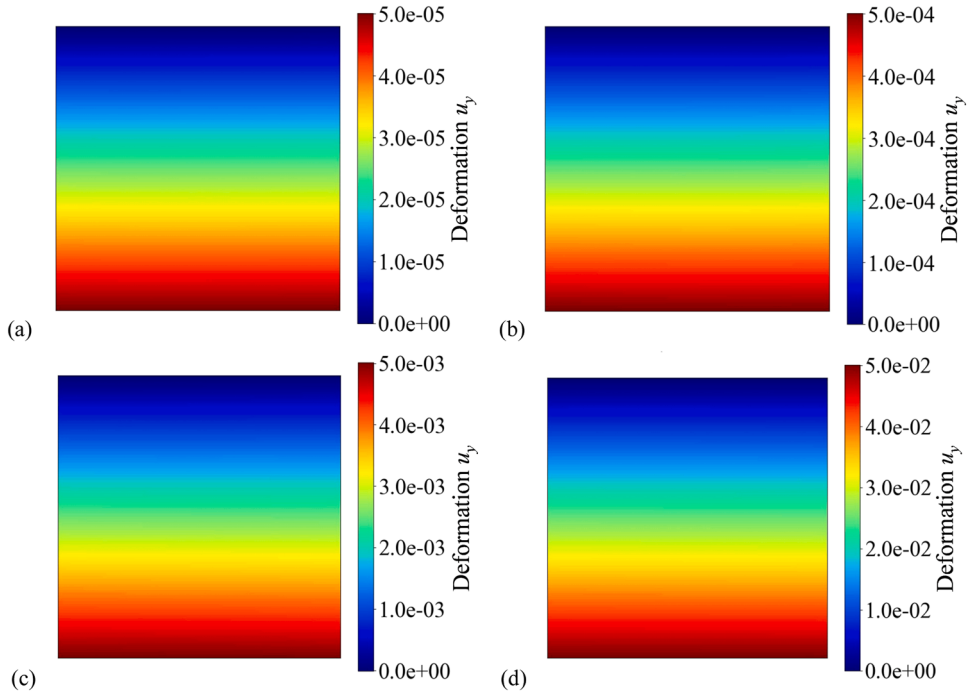


Fig. 14. Contours of vertical deformation u_y, (a) $p = 10^0$ kPa, (b) $p = 10^1$ kPa, (c) $p = 10^2$ kPa, (d) $p = 10^3$ kPa.

5. Conclusion

The novel unsupervised PINN framework presented in this study marks a significant advancement in the application of PINN for solid mechanics by effectively addressing the critical issue of training imbalance. The convergence and accuracy of the multi-task PINN model are enhanced through the integration of dynamically self-adaptive weights, updated using either the modified GradNorm algorithm or the augmented Lagrangian algorithm, within a newly formulated loss function.

The self-adaptive weight updating strategy not only improves the predictive accuracy but also facilitates the training efficiency. The results indicate that the PINN model performs optimally with specific neural network architectures and training data, when learning rates between 1.0×10^{-3} and 3.0×10^{-3}, data point spacing of 1.0×10^{-2} to 4.0×10^{-2} m, and architectures with 4 to 10 layers and 40 to 100 neurons.

Furthermore, an information entropy-based approach is developed to evaluate the uncertainty inherent in the PINN models. The average information entropy of prediction errors, quantified from the well-trained 500 models, is 1.31 bits when the threshold of convergence criterion is set at 1×10^{-11}. This low information entropy implies high prediction accuracy and low uncertainty, confirming the robustness and reliability of the developed PINN model.

The findings also uncover a "squared rule", where the reduction factor of the predefined threshold is approximately the square of the load increase factor. This novel framework makes a valuable contribution to solving the multitask imbalance issue, and the "squared rule" provides important insights for selecting the threshold of convergence criterion, paving the way for future

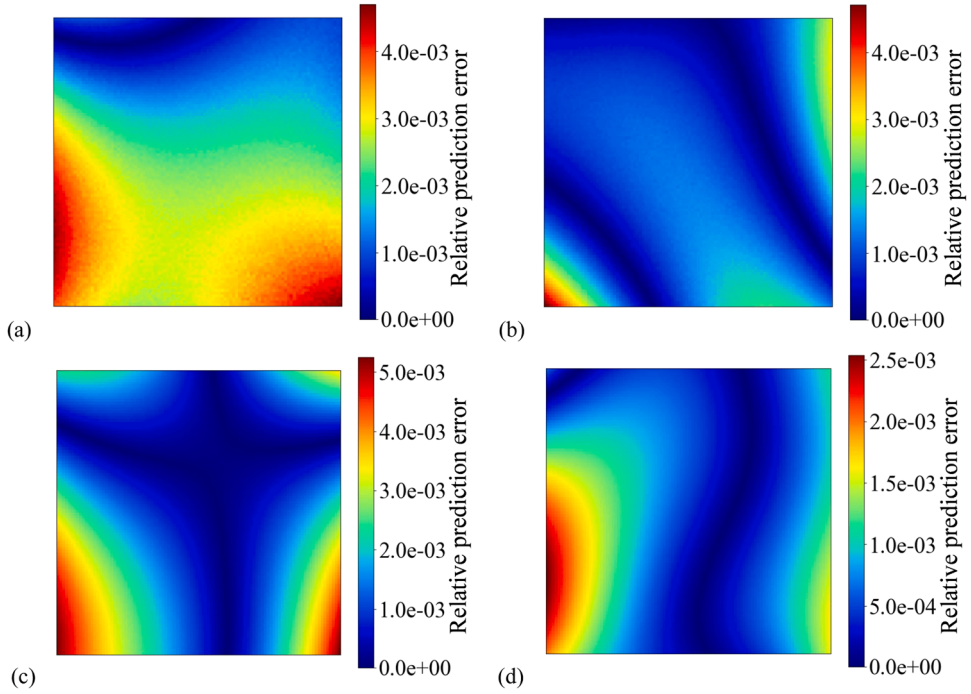


Fig. 15. Relative prediction error contours of vertical deformation, (a) $p = 10^0$ kPa, (b) $p = 10^1$ kPa, (c) $p = 10^2$ kPa, (d) $p = 10^3$ kPa.

developments in unsupervised learning PINN for solid mechanics.

Building upon the demonstrated success in linear elasticity, future work will aim to extend this framework to nonlinear elasticity, elastic-plasticity, and coupled multi-physics problems, further broadening its applicability and impact in real-world engineering contexts.

CRediT authorship contribution statement

Feiyang Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Wuzhou Zhai:** Writing – review & editing, Resources, Investigation, Formal analysis, Conceptualization. **Shuai Zhao:** Writing – review & editing, Investigation, Formal analysis. **Jianhong Man:** Writing – review & editing, Validation, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This study is substantially sponsored by the National Natural Science Foundation of China (Grant No. 52308408), the Fundamental Research Funds for the Central Universities (Grant No. 2232024D-16), and the Science and Technology Commission of Shanghai Municipality, China (Grant No. 22YF1429900). The authors are grateful to these projects.

Data availability

The data that has been used is confidential.

References

[1] M. Abadi, A. Agarwal, P. Barham, et al., Tensorflow: Large-scale machine learning on heterogeneous distributed systems[J], arXiv preprint arXiv:1603.04467, 2016.
[2] G E Karniadakis, I G Kevrekidis, L. Lu, et al., Physics-informed machine learning, Nat. Rev. Phys. 3 (2021) 422–440.
[3] S. Zhao, F Y Wang, D Y Tan, et al., A deep learning informed-mesoscale cohesive numerical model for investigating the mechanical behavior of shield tunnels with crack damage[J], Structures 66 (2024) 106902.

[4] S H Rudy, S L Brunton, J L Proctor, et al., Data-driven discovery of partial differential equations[J], Sci. Adv. 3 (4) (2017) e1602614.
[5] B. Lusch, J N Kutz, S L Brunton, Deep learning for universal linear embeddings of nonlinear dynamics[J], Nat. Commun. 9 (1) (2018) 4950.
[6] Y. Wei, X. Zhang, Y. Shi, et al., A review of data-driven approaches for prediction and classification of building energy consumption[J], Renew. Sustain. Energy Rev. 82 (2018) 1027–1047.
[7] M. Raissi, A. Yazdani, G E Karniadakis, Hidden fluid mechanics: learning velocity and pressure fields from flow visualizations[J], Science 367 (6481) (2020) 1026–1030.
[8] J. Bai, H. Jeong, C P Batuwatta-Gamage, et al., An introduction to programming physics-informed neural network-based computational solid mechanics[J], arXiv preprint arXiv:2210.09060, 2022.
[9] S. Rezaei, A. Harandi, A. Moeineddin, et al., A mixed formulation for physics-informed neural networks as a potential solver for engineering problems in heterogeneous domains: comparison with finite element method[J], Comput. Methods Appl. Mech. Eng. 401 (1) (2022) 350–383.
[10] W. Wu, M. Daneker, M A Jolley, et al., Effective data sampling strategies and boundary condition constraints of physics-informed neural networks for identifying material properties in solid mechanics[J], Appl. Math. Mech. (Engl. Ed.) 44 (7) (2023) 1039–1068.
[11] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations[J], J. Comput. Phys. 378 (2019) 686–707.
[12] C. Zheng, Y. Wang, S. Chen, Data-driven constitutive relation reveals scaling law for hydrodynamic transport coefficients[J], Phys. Rev. E 107 (1) (2023) 015104.
[13] F. Wang, W. Zhai, J. Man, et al., A hybrid cohesive phase-field numerical method for the stability analysis of rock slopes with discontinuities[J], Can. Geotech. J. 62 (2025) 1–16.
[14] T. Zhang, Y. Zhang, K. Katterbauer, et al., Deep learning–assisted phase equilibrium analysis for producing natural hydrogen[J], Int. J. Hydrog. Energy 50 (2024) 473–486.
[15] X. Feng, J. Liu, J. Shi, et al., Phase equilibrium, thermodynamics, hydrogen-induced effects and the interplay mechanisms in underground hydrogen storage[J], Comput. Energy Sci. 1 (1) (2024) 46–64.
[16] T G Grossmann, U J Komorowska, J. Latz, et al., Can physics-informed neural networks beat the finite element method?[J], arXiv preprint arXiv:2302.04107, 2023.
[17] I E Lagaris, A. Likas, D I Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.
[18] S. Subramanian, R M Kirby, M W Mahoney, et al., Adaptive self-supervision algorithms for physics-informed neural networks[J], arXiv preprint arXiv: 2207.04084, 2022.
[19] L. Lu, R. Pestourie, W. Yao, et al., Physics-informed neural networks with hard constraints for inverse design, SIAM J. Sci. Comput. 43 (6) (2021) B1105–B1132.
[20] W. Wu, M. Daneker, M A Jolley, et al., Effective data sampling strategies and boundary condition constraints of physics-informed neural networks for identifying material properties in solid mechanics[J], Appl. Math. Mech. 44 (7) (2023) 1039–1068.
[21] S. Liu, Z. Hao, C. Ying, et al., A unified hard-constraint framework for solving geometrically complex PDEs[J], Adv. Neural Inf. Process. Syst. 35 (2022) 20287–20299.
[22] E. Schiassi, R. Furfaro, H M D Leake, Extreme theory of functional connections: A fast physics-informed neural network method for solving ordinary and partial differential equations[J], Neurocomputing 457 (10) (2021) 334–356.
[23] C. Leake, D. Mortari, Deep theory of functional connections: A new method for estimating the solutions of partial differential equations[J], Mach. Learn. Knowl. Extr. 2 (1) (2020) 37–55.
[24] D. Mortari, The Theory of Connections. Connecting Points[J], Mathematics 5 (4) (2017) 57.
[25] E. Haghighat, M. Raissi, A. Moure, et al., A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics[J], Comput. Methods Appl. Mech. Eng. 379 (2021) 113741.
[26] J. Yao, C. Su, Z. Hao, et al., Multiadam: Parameter-wise scale-invariant optimizer for multiscale training of physics-informed neural networks[J], arXiv preprint arXiv:2306.02816, 2023.
[27] M. Guo, A. Haque, D A Huang, et al., Dynamic task prioritization for multitask learning, in: [C]//Proceedings of the European conference on computer vision (ECCV), 2018, pp. 270–287.
[28] A. Kendall, Y. Gal, R. Cipolla, Multi-task learning using uncertainty to weigh losses for scene geometry and semantics[C], in: Proceedings of the IEEE conference on computer vision and pattern recognition, 2018, pp. 7482–7491.
[29] S. Liu, E. Johns, A J Davison, End-to-end multi-task learning with attention[C], in: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, 2019, pp. 1871–1880.
[30] Z. Chen, V. Badrinarayanan, C Y Lee, et al., Gradnorm: gradient normalization for adaptive loss balancing in deep multitask networks[C], in: International conference on machine learning, PMLR, 2018, pp. 794–803.
[31] Y. He, X. Feng, C. Cheng, et al., Metabalance: improving multi-task recommendations via adapting gradient magnitudes of auxiliary tasks, in: [C]//Proceedings of the ACM Web Conference 2022, 2022, pp. 2205–2215.
[32] C. Xu, B T Cao, Y. Yuan, et al., Transfer learning based physics-informed neural networks for solving inverse problems in engineering structures under different loading scenarios[J], Comput. Methods Appl. Mech. Eng. 405 (2023) 115852.
[33] J. Bai, T. Rabczuk, A. Gupta, et al., A physics-informed neural network technique based on a modified loss function for computational 2D and 3D solid mechanics[J], Comput. Mech. 71 (2023) 543–562.
[34] L. Lu, R. Pestourie, W. Yao, et al., Physics-informed neural networks with hard constraints for inverse design[J], SIAM J. Sci. Comput. 43 (6) (2021) B1105–B1132.
[35] E. Samaniego, C. Anitescu, S. Goswami, et al., An energy approach to the solution of partial differential equations in computational mechanics via machine learning: concepts, implementation and applications[J], Comput. Methods Appl. Mech. Eng. 362 (2020) 112790.
[36] D P Kingma, J. Ba, Adam: A method for stochastic optimization[J], arXiv preprint arXiv:1412.6980, 2014.
[37] R H Byrd, P. Lu, J. Nocedal, et al., A limited memory algorithm for bound constrained optimization[J], SIAM J. Sci. Comput. 16 (5) (1995) 1190–1208.
[38] Z. Mao, A D Jagtap, G.E Karniadakis, Physics-informed neural networks for high-speed flows[J], Comput. Methods Appl. Mech. Eng. 360 (2020) 112789.
[39] S. Cuomo, V S Di Cola, F. Giampaolo, et al., Scientific machine learning through physics–informed neural networks: where we are and what's next[J], J. Sci. Comput. 92 (3) (2022) 88.
[40] J F Wellmann, K. Regenauer-Lieb, Uncertainties have a meaning: information entropy as a quality measure for 3-D geological models[J], Tectonophysics 526 (2012) 207–216.
[41] D W Scott, Sturges' rule[J], Wiley Interdiscip. Rev.: Comput. Stat. 1 (3) (2009) 303–306.